

FinSight — Final Year Project Viva Study Guide

This guide is structured specifically for Student A (Web Dev background) and Student B (Non-programmer background) for the FinSight project viva. It focuses on **where code lives**, **how data flows**, and **how to explain key concepts out loud**.

1. Concept Primers

FastAPI (Backend Framework)

FastAPI is a high-performance Python web framework used to build RESTful APIs. It automatically validates incoming data against Pydantic schemas, handles asynchronous execution, and generates interactive OpenAPI documentation at **/docs**. In FinSight, FastAPI routes incoming HTTP requests to business service functions and returns structured JSON responses.

- **Request Flow:** User submits form → Frontend `axios.post('/api/transactions')` → FastAPI endpoint validates payload via `TransactionCreate` schema → calls `transaction_service.create_transaction()` → SQLAlchemy writes record to PostgreSQL database → FastAPI returns HTTP 201 with serialized transaction JSON.
- **Real Endpoint Example:** `backend/app/api/routes/transactions.py` function `create_transaction()`.

Pydantic Schema vs. SQLAlchemy Model

- **Pydantic Schema:** Operates in memory at the API layer. It defines data shapes, types, and validation rules for incoming API request bodies and outgoing responses (e.g., checking if amount > 0 or verifying email format).
- *Real Example:* `TransactionCreate` in `backend/app/schemas/transaction.py` defines validated fields like `amount: Decimal = Field(gt=0)` and `type: Literal['income', 'expense', 'transfer']`.
- **SQLAlchemy Model:** Operates at the database layer. It maps Python classes directly to PostgreSQL relational tables, defining column data types, foreign key relationships, primary keys, and index constraints.
- *Real Example:* `Transaction` in `backend/app/models/transaction.py` maps to the `transactions` table with SQL columns (`id`, `user_id`, `account_id`, `amount`, `deleted_at`).

Database Migrations

A database migration applies incremental structural changes (like adding columns or tables) to an existing database without destroying existing data. FinSight uses a lightweight, native SQL migration script with **IF NOT EXISTS** queries in `backend/app/core/migrate.py`.

- **How It Runs:** FinSight automatically triggers **run_migrations(engine)** on backend startup inside **backend/app/main.py**.
- **Migration History / Structure:** Located in **backend/app/core/migrate.py**. Executes idempotent SQL commands like **ALTER TABLE transactions ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMP** and creates missing tables like **accounts**.

React (Frontend UI Library)

React builds dynamic user interfaces using modular components.

- **Components:** Reusable UI building blocks defined as functions returning JSX (HTML combined with JS).
- *Real Example:* **TransactionTable.jsx**.
- **Props:** Read-only inputs passed from parent to child components (e.g., passing **transactions** array to **TransactionTable**).
- **State:** Internal dynamic data managed by hooks like **useState()** or Redux store that causes the component to re-render when altered.
- **Hooks:** Special functions starting with **use** (e.g., **useEffect**, **useAuth**) that hook into React lifecycle and state features.

TailwindCSS (Styling)

TailwindCSS is a utility-first CSS framework that provides low-level atomic classes (like **flex**, **pt-4**, **bg-indigo-600**, **rounded-xl**) directly inside JSX **className** attributes. Unlike plain CSS which requires separate **.css** files and class rule declarations (**.my-btn { background: blue; }**), Tailwind applies styles inline in the markup, ensuring consistent design tokens and zero unused CSS bundle bloat.

PostgreSQL Database & High-Level Schema

PostgreSQL is an enterprise-grade relational database chosen over NoSQL for strict ACID compliance, relational integrity (foreign key constraints), and strong aggregation capabilities for monetary calculations.

• Database Tables:

1. **users** (authentication credentials & profile settings)
2. **accounts** (cash, bank accounts, credit cards)
3. **transactions** (income, expense, and transfer records)
4. **budgets** (monthly spending limits per category)
5. **goals** (savings targets)
6. **goal_contributions** (deposits made towards specific goals)
7. **recurring_bills** (scheduled subscriptions or regular bills)
8. **user_category** (custom categories created by users)
9. **networth_snapshots** (historical wealth tracking)
10. **dismissed_notifications** (user alert history)

REST API Architecture

REST (Representational State Transfer) defines standardized HTTP verb conventions (**GET**, **POST**, **PUT**, **DELETE**) for client-server communication over URL routes. In FinSight, frontend API functions in `frontend/src/api/` use **axios** to make HTTP calls to FastAPI endpoints (e.g., `api.post('/transactions', data)`), which process data on PostgreSQL and return standard JSON payloads with HTTP status codes (**200 OK**, **201 Created**, **400 Bad Request**, **404 Not Found**).

2. Project Structure Map

Backend Directory Map (backend/app/)

Directory / File	Purpose & Contents	Why Organized This Way
<u>models/</u>	SQLAlchemy table schemas (<code>transaction.py</code> , <code>account.py</code> , <code>user.py</code> , etc.)	Isolates raw PostgreSQL database table declarations.
<u>schemas/</u>	Pydantic validation models (<code>transaction.py</code> , <code>budget.py</code> , <code>auth.py</code>)	Separates API request/response validation contracts from DB models.
<u>api/routes/</u>	FastAPI router endpoints (<code>transactions.py</code> , <code>accounts.py</code> , <code>auth.py</code>)	Thin HTTP handler layer: handles status codes, auth headers, and routing.
<u>services/</u>	Core business logic and mathematical algorithms (<code>categorizer_service.py</code> , <code>anomaly_service.py</code> , <code>account_service.py</code>)	Decouples business logic from HTTP endpoints for modularity and testability.
<u>core/</u>	App configuration (<code>config.py</code>), DB engine setup (<code>database.py</code>), JWT security (<code>security.py</code>), and migrations (<code>migrate.py</code>)	Centralizes global infrastructure dependencies.
<u>main.py</u>	Application entry point: initializes FastAPI app, CORS, and registers API routers	Single entry point for ASGI web server (Uvicorn).

Frontend Directory Map (frontend/src/)

Directory / File	Purpose & Contents	Why Organized This Way
<u>pages/</u>	Page-level components (<code>DashboardPage.jsx</code> , <code>TransactionsPage.jsx</code> , <code>GoalsPage.jsx</code>)	Top-level view containers rendered by React Router.
<u>components/</u>	Reusable visual components grouped by feature (<code>budget/</code> , <code>charts/</code> , <code>transactions/</code> , <code>ui/</code>)	Facilitates code reuse across pages.
<u>store/</u>	Redux Toolkit slices (<code>authSlice.js</code> , <code>transactionSlice.js</code>)	Centralizes global application state.

<u>api/</u>	Axios client modules (transactions.js , auth.js)	Encapsulates HTTP network calls to the FastAPI backend.
<u>hooks/</u>	Custom React hooks (useAuth.js , useTransactions.js)	Encapsulates shared stateful logic across components.
<u>constants/</u>	Static data constants (categories.js , colors.js)	Eliminates magic strings across frontend views.

3. Feature-to-Code Map

Transactions CRUD [EASY]

- **Backend Files:** [backend/app/api/routes/transactions.py](#), [backend/app/services/transaction_service.py](#), [backend/app/models/transaction.py](#)
- **Frontend Files:** [frontend/src/pages/TransactionsPage.jsx](#), [frontend/src/components/transactions/TransactionTable.jsx](#), [frontend/src/components/transactions/TransactionForm.jsx](#)
- **Data Flow:** User completes form in **TransactionForm.jsx** → sends **POST /api/transactions** → validated by **TransactionCreate** schema → **transaction_service.create_transaction()** attaches **account_id** and saves row → returns serialized object → Redux store updates state → **TransactionTable.jsx** re-renders table row.
- **Tricky Logic:** Uses **Soft Delete (deleted_at timestamp)**. Deleting a transaction does not purge the database row; it marks **deleted_at = NOW()**, allowing transaction restoration or standard trash retention.

Accounts & Balance Calculation [CORE]

- **Backend Files:** [backend/app/api/routes/accounts.py](#), [backend/app/services/account_service.py](#), [backend/app/models/account.py](#)
- **Frontend Files:** [frontend/src/pages/AccountsPage.jsx](#), [frontend/src/pages/AccountDetailPage.jsx](#)
- **Data Flow:** Account balance is computed dynamically on demand. Frontend requests **GET /api/accounts** → backend calls **account_service._balance()** for each account → sums transactions by type → returns computed current balance.
- **Tricky Logic:** Account balance is calculated dynamically using the formula:

Balance = Opening Balance + Income - Expenses - Transfers Out + Transfers In

Deleting an account safely reassigns all its associated transactions to the user's default Cash account before removing the account.

Budgets & Pace Prediction [CORE]

- **Backend Files:** [backend/app/api/routes/budgets.py](#), [backend/app/services/budget_service.py](#), [backend/app/services/pace_service.py](#)

- **Frontend Files:** [frontend/src/pages/BudgetsPage.jsx](#), [frontend/src/components/budget/BudgetCard.jsx](#)
- **Data Flow:** Users set monthly category limits. When viewing budgets, **budget_service** aggregates current month's expenses for that category and invokes **compute_budget_pace()** to calculate spending trajectory.
- **Tricky Logic: Rollover & Pace Projection:**
 - If **rollover** is enabled, unused funds from previous months carry over up to 24 months recursively.
 - Pace prediction formula: $\text{Daily Rate} = \frac{\text{Spent} - \text{Current Day}}{\text{Projected Spend} - \text{Current Day}}$, $\text{Projected Spend} = \text{Daily Rate} \times \text{Days in Month}$. Status is classified as **on_track**, **at_risk** (>80% projected), **will_exceed** (>100% projected), or **exceeded** (already spent > limit).

Goals & Contributions [EASY]

- **Backend Files:** [backend/app/api/routes/goals.py](#), [backend/app/services/goal_service.py](#)
- **Frontend Files:** [frontend/src/pages/GoalsPage.jsx](#), [frontend/src/components/goals/GoalProgressRing.jsx](#)
- **Data Flow:** User creates a goal with a target amount. Depositing money creates a **goal_contribution** record and an automated transfer transaction from the user's primary account to a dedicated "Goal savings" system account.
- **Tricky Logic:** Automatically provisions a "Goal savings" account in the **accounts** table on first deposit so saved funds are excluded from liquid cash balance without deleting the money.

Recurring Bills [EASY]

- **Backend Files:** [backend/app/api/routes/recurring_bills.py](#), [backend/app/services/recurring_bill_service.py](#)
- **Frontend Files:** [frontend/src/pages/BillsPage.jsx](#)
- **Data Flow:** Manages subscriptions and regular obligations (weekly/monthly/yearly). Clicking "Record Payment" converts the bill into an active transaction record.
- **Tricky Logic:** Normalizes subscription costs to monthly equivalents ($\text{weekly spend} \times 52 / 12$, $\text{annual spend} \times 1 / 12$) for accurate monthly expense forecasting.

Categories Management [EASY]

- **Backend Files:** [backend/app/api/routes/categories.py](#), [backend/app/models/user_category.py](#)
- **Frontend Files:** [frontend/src/pages/CategoriesPage.jsx](#)
- **Data Flow:** Provides system default categories (food, transport, housing, etc.) alongside custom user-defined categories saved in **user_category**.

Authentication & Authorization [EASY]

- **Backend Files:** [backend/app/api/routes/auth.py](#), [backend/app/services/auth_service.py](#), [backend/app/core/security.py](#)
- **Frontend Files:** [frontend/src/pages/LoginPage.jsx](#), [frontend/src/features/auth/authSlice.js](#)

- **Data Flow:** Registration hashes raw passwords using **bcrypt**. Login verifies credentials and returns a signed JWT access token. Frontend stores token in **localStorage** and includes **Authorization: Bearer** header on every request. **get_current_user** dependency in FastAPI extracts user context from token payload.

Algorithm 1: Naive Bayes Categorizer + Levenshtein Typo Matching [CORE]

- **File Path:** **backend/app/services/categorizer_service.py**
- **Problem Solved:** Automatically predicts and suggests transaction category in real-time as user types transaction description (e.g. "pathao ride to office" → **transport**), adapting to typos and learning from user history.

- **Mathematical & Logical Explanation:**

1. **Brand & Phrase Shortcut:** First checks description against static lookup **PHRASE_HINTS** (e.g. "pathao", "bhatbhateni").
2. **Levenshtein Distance:** For unknown terms, measures min edit distance (insertions, deletions, substitutions) between token a and vocabulary term b:

$$d(a, b) = \min(\text{del}, \text{ins}, \text{sub})$$

Matches typos (e.g. "pathao" → "pathao" with distance=1).

3. **Multinomial Naive Bayes Classification:** Computes posterior probability of category C given word tokens W:

$$P(C \mid W) \propto P(C) \prod_{i=1}^n P(w_i \mid C)$$

Uses **Laplace Smoothing** to prevent zero-probability errors for unseen words:

$$P(w_i \mid C) = (\text{count}(w_i, C) + 1) / (\sum \text{count}(w, C) + |V| + 1)$$

Executes calculations in log-space ($\log P(C) + \sum \log P(w_i \mid C)$) to prevent floating-point underflow.

- **Why Chosen Over Alternatives:** Requires zero external ML dependencies, trains instantly in memory on per-user data, and operates in < 5 milliseconds without GPU resources.
- **Numerical Step-by-Step Walkthrough:**
 - Input: **"pathao ride"** (Type: **expense**)
 - Tokenized: **["pathao", "ride"]**
 - Levenshtein match: **"pathao" → "pathao"** (distance = 1).
 - Remapped tokens matched to vocabulary seed words in **transport**.
 - Bayes probability output: **transport** category returned with **85.0% confidence**.
- **Known Edge Cases & Solutions:**
 - *Edge Case:* Description "Car sold" contains token "car" which strongly correlates to **transport**, but is actually **salary/income**.
 - *Handling Solution:* Categorizer restricts valid outcome sets based on transaction type (**VALID_EXPENSE** vs **VALID_INCOME**). Furthermore, predictions with confidence < 28% are rejected and return **None**.

Algorithm 2: 1D K-Means Spending Clustering [CORE]

- **File Path:** backend/app/services/clustering_service.py

- **Problem Solved:** Groups user's expenses into 3 natural spending tiers (**micro**: small buys, **regular**: daily spend, **major**: big purchases) without pre-defined fixed threshold values.

- **Mathematical & Logical Explanation:**

1. Sort transaction amounts. Select initial centroids μ_0, μ_1, μ_2 as the minimum, median, and maximum amounts.

2. **Assignment Step:** Assign each expense amount x_i to nearest centroid k :

$$\text{Cluster}(x_i) = \arg \min_k |x_i - \mu_k|$$

3. **Update Step:** Recalculate centroids as the arithmetic mean of assigned points:

$$\mu_k = \frac{1}{|S_k|} \sum_{x \in S_k} x$$

4. Iterate until centroid assignments stabilize or reaching maximum 100 iterations.

- **Why Chosen Over Alternatives:** Simple, unsupervised algorithm that adapts dynamically to individual wealth levels (a student's "major expense" might be Rs. 3,000, whereas a high-earner's might be Rs. 50,000).

- **Numerical Step-by-Step Walkthrough:**

- Expense amounts: **[100, 150, 200, 1500, 1800, 10000]**

- Initial Centroids: $\mu_0 = 100, \mu_1 = 175, \mu_2 = 10000$

- Iteration 1 Assignments:

- Cluster 0 (**micro**): **[100, 150, 200]** → New $\mu_0 = 150.0$

- Cluster 1 (**regular**): **[1500, 1800]** → New $\mu_1 = 1650.0$

- Cluster 2 (**major**): **[10000]** → New $\mu_2 = 10000.0$

- Centroids converge in 2 iterations.

- **Known Edge Cases & Solutions:**

- *Edge Case:* User has <3 transactions or duplicate initial centroid values.

- *Handling Solution:* If <3 transactions, sets static min/max centroids without iteration and sets **insufficient_data: true**. If initial centroids are duplicate, adds small scalar offset (**+1.0**) to force separation.

Algorithm 3: Z-Score Anomaly Detection [CORE]

- **File Path:** backend/app/services/anomaly_service.py

- **Problem Solved:** Flags unusually high or low expenses compared to historical spending habits per category (e.g. "Usually spend Rs. 500 on Food, today spent Rs. 5,000").

- **Mathematical & Logical Explanation:**

1. Retrieve historical expense list X for the specific category.

2. Compute historical Mean (μ) and Variance (σ^2):

$$\mu = \frac{1}{N} \sum x_i, \quad \sigma = \sqrt{\frac{1}{N} \sum (x_i - \mu)^2}$$

3. **Epsilon Floor:** Prevents division by zero when historical expenses are identical:

$$\sigma_{\text{floored}} = \max(\sigma, |\mu| \times 0.05)$$

4. **Calculate Z-Score:**

$$Z = (x_{\text{new}} - \mu) / \sigma_{\text{floored}}$$

5. **Severity Classification:** $|Z| \geq 3 \rightarrow \text{"high"}$, $|Z| \geq 2 \rightarrow \text{"moderate"}$.

- **Why Chosen Over Alternatives:** Standardized statistical measure that provides clean, intuitive human-readable explanations ("This is 3.5x higher than your average").
- **Numerical Step-by-Step Walkthrough:**
 - Past Food expenses: **[400, 450, 500, 550, 600]** (N=5)
 - Mean $\mu = 500.0$, Standard Deviation $\sigma \approx 70.71$
 - New transaction: Rs. 1,200 Food expense.
 - Calculate Z-Score: $Z = (1200 - 500) / 70.71 = 9.90$
 - Result: Flagged as **HIGH severity anomaly** ($Z \geq 3.0$).
- **Known Edge Cases & Solutions:**
 - *Edge Case:* Category has <5 transactions (statistical noise) or user repeatedly sees false alerts for valid big purchases.
 - *Handling Solution:* Requires **MIN_SAMPLES = 5**; returns **status: "not_enough_data"** if history <5. Users can click "Acknowledge", setting **anomaly_acknowledged = True** in DB to permanently suppress the alert.

4. Difficulty Split & Talking Points

Student A (Web Dev Background) — CORE Topics Owner

- **Assigned Systems:** All 3 Algorithms, Dynamic Balance Logic, Budget Pace Math, JWT Security, Database Architecture.
- **Suggested Viva Talking Points:**
 1. "I implemented the statistical and machine learning algorithms in **backend/app/services/**."
 2. "Our Naive Bayes classifier uses Laplace smoothing and log-probabilities to categorize transactions from description text, supported by Levenshtein distance for fuzzy typo matching."
 3. "Account balances are dynamically aggregated in real-time from ledger entry types (income, expense, transfer) to guarantee consistency without stale counter columns."
 4. "Our anomaly detection uses a modified Z-score formula with an epsilon floor (mean $\times 0.05$) to prevent division by near-zero standard deviations."

Student B (Non-Programmer Background) — EASY Topics Owner

- **Assigned Systems:** Feature locations, UI Data Flows, CRUD Operations, User Authentication concept, Basic Component mapping.

• Suggested Viva Talking Points:

1. "I can demonstrate our application layout and explain how data flows from user input forms to our database tables."
 2. "When a user adds a transaction in **TransactionForm.jsx**, it sends a **POST** request to our FastAPI server, which validates the fields before saving it to PostgreSQL."
 3. "Our system supports budget tracking, recurring bills, and savings goals. For savings goals, contributions automatically create transfer records that put funds into a dedicated Goal Savings account."
 4. "We use PostgreSQL as our database with tables for users, accounts, transactions, budgets, and goals, linked together using foreign keys."
-

5. Likely Viva Questions & Answers

Q1: Why FastAPI instead of Flask or Django?

Answer: FastAPI provides automatic request data validation via Pydantic, built-in interactive OpenAPI docs (**/docs**), high performance with async support, and a lightweight footprint compared to heavy monolithic frameworks like Django.

Q2: What is the exact difference between a Pydantic schema and a SQLAlchemy model?

Answer: Pydantic schemas operate in memory at the API boundary to validate incoming HTTP request payloads and format JSON responses. SQLAlchemy models define the database schema and handle Object-Relational Mapping (ORM) to read/write PostgreSQL table rows.

Q3: How do you compute an account balance? Why not store balance as a column in the database?

Answer: Storing balance directly risks data drift if a transaction is updated or soft-deleted. We compute balance dynamically using: Opening Balance + Income - Expenses - Transfers Out + Transfers In in **account_service.py**.

Q4: Walk me through what happens end-to-end when a user creates a transaction.

Answer: User fills **TransactionForm.jsx** → Axios sends **POST /api/transactions** → FastAPI validates payload with **TransactionCreate** schema → **transaction_service.create_transaction()** attaches default account & checks anomalies → SQLAlchemy inserts row into PostgreSQL **transactions** table → returns JSON to frontend → Redux store updates and table updates.

Q5: Why did you choose Naive Bayes instead of a Deep Learning / Neural Network model for categorization?

Answer: Naive Bayes is lightweight, fast, computes predictions in <5ms in memory, requires no GPU or external model server, and performs exceptionally well on text classification even with small training datasets.

Q6: What happens in Anomaly Detection if a category only has 2 past transactions?

Answer: Anomaly detection is skipped. The service enforces **MIN_SAMPLES = 5**. Calculating mean and standard deviation on fewer than 5 items produces statistically invalid results and false alarms.

Q7: Explain the purpose of the Epsilon Floor in your Z-score anomaly detection.

Answer: If a user pays identical amounts (e.g. Rs. 500 rent 5 times), the standard deviation (σ) becomes 0. Dividing by zero causes a crash or infinite Z-score. We floor σ at $\max(\sigma, \text{mean} \times 0.05)$ to maintain statistical stability.

Q8: How does K-Means clustering initialize centroids in your implementation?

Answer: We sort user expense amounts and set initial centroids to the minimum, median, and maximum values. If initial centroids overlap, we add a scalar offset (**+1.0**) to force distinct cluster boundaries.

Q9: How are database schema changes handled without Alembic?

Answer: We use an idempotent SQL migration function in **backend/app/core/migrate.py** that runs automatically on FastAPI server startup, utilizing **ALTER TABLE ... ADD COLUMN IF NOT EXISTS** and **CREATE TABLE IF NOT EXISTS**.

Q10: What is soft deletion and why is it used for transactions?

Answer: Instead of running SQL **DELETE**, soft deletion sets a **deleted_at = NOW()** timestamp column. This preserves historical records for audits, budget reconciliations, and undo functionality while filtering them out of standard active queries.

Q11: How does JWT authentication protect backend routes?

Answer: Upon login, the server returns a signed JSON Web Token (JWT). The frontend attaches this token to the **Authorization: Bearer** header. Backend routes depend on **get_current_user** in **deps.py**, which verifies token signature and extracts **user_id**.

Q12: How does budget pace prediction calculate if a user will exceed their budget?

Answer: It computes daily burn rate: $\text{Daily Rate} = (\text{Spent} / \text{Current Day of Month})$, then projects end-of-month spend: $\text{Projected} = \text{Daily Rate} \times \text{Total Days in Month}$. If projected spend exceeds limit, status is marked **will_exceed**.